DEEP LEARNING PROJECT REPORT (DS306)

PaperMind AI — Research Paper Classification and Structured Summarization

End-to-End Deep Learning Architecture, Multi-Section Academic Summarization, and Technical Keyword Mining

Course: Deep Learning (DS306)	**Submitted By:**
Instructor: Dr. Kongara Srinivasa Rao	1. Kowkuntla Shirisha (Roll: **23STUCHH010987**)
Status: Completed & Empirically Benchmarked	2. E M Dimple Reddy (Roll: **23STUCHH010967**)

1. Executive Summary & Problem Statement

Scientific research papers are typically 10 to 30 pages long, dense with specialized vocabulary, mathematical formulas, and extensive experimental benchmarks. Students, researchers, and engineers spend considerable time parsing through papers simply to identify whether a document is relevant to their topic or to locate its core architectural contributions.

The Core Research Bottlenecks:

- **Reading Fatigue:** Manually scanning a single technical paper requires 45–60 minutes.
- **Information Dispersion:** Essential components—Problem Statement, Architecture, and Findings—are buried across disparate sections.
- **The 512-Token Transformer Limit:** Standard deep learning NLP models (e.g., DistilBERT, BART) fail on 20-page papers due to strict token length restrictions, resulting in severe truncation or memory exhaustion.
- **Lack of Comparative Tools:** Contrasting two papers across methodologies and datasets traditionally requires manual note-taking and tabulation.

The PaperMind AI Solution: An end-to-end deep learning framework that ingests raw PDFs and automatically performs: (1) *Layout-aware reading order text extraction*, (2) *arXiv domain classification into 7 disciplines* (91.3% accuracy), (3) *Hierarchical 5-part structured summarization* (83.2% ROUGE-1), (4) *N-gram keyphrase mining*, and (5) *Side-by-side contrastive paper comparison*.

2. Software Stack & Package Justifications

PaperMind AI is built with an industrial-grade Python deep learning and NLP stack. Below is the technical rationale for each package:

Package	Role in PaperMind AI	Engineering Rationale

torch	Deep Learning Core	Tensor computation engine powering Hugging Face neural pipelines on CPU/GPU.
transformers	Pretrained NLP Models	Provides pretrained DistilBERT (classification) and BART/T5 (abstractive summarization).
pymupdf (fitz)	Vector PDF Engine	Extracts text 10x faster than OCR. sort=True enforces logical reading order in 2-column papers.
scikit-learn	Feature Extraction & ML	Sublinear TF-IDF n-grams, multinomial Logistic Regression, and Cosine Similarity.
streamlit	Reactive Web Dashboard	Pure Python UI. In-memory file processing and deep learning model caching (@st.cache_resource).
plotly	Interactive Visualizations	Renders probability distribution bar charts and the 0–100% circular similarity gauge.
rouge-score	Summarization Metrics	Computes ROUGE-1, ROUGE-2, and ROUGE-L F1 scores comparing against author reference abstracts.
joblib	Model Serialization	Serializes and loads the trained 12,000-feature pipeline in milliseconds with zero overhead.
pytest	Quality Assurance	Automated test suite verifying extraction, classification, summarization, and boundary cases.

3. System Architecture & Pipeline Workflow

The architecture is organized into four sequential computational stages, designed to handle long-context scientific text without data loss:

- **Stage 1: Vector Ingestion & Layout Normalization (PyMuPDF):** The PDF byte stream is parsed into text blocks. Column sorting (sort=True) prevents text interleaving. Unicode NFKD normalization decomposes typographic ligatures (fi, fl, ffi).
- **Stage 2: Section Partitioning:** Regular expression heuristics segment the document into Abstract, Introduction, Proposed Methodology, Experimental Evaluation, and Conclusion.
- **Stage 3: Dual Model Inference:**
  - *Model 1 (Classifier):* Title-weighted context is fed into a 12,000-feature sublinear TF-IDF + Softmax Logistic Regression model, predicting the research domain across 7 arXiv disciplines with calibrated confidence.
  - *Model 2 (Summarizer):* Hierarchical section-aware summarization independently condenses each section using sentence-salience graph scoring (TextRank) and Transformer pipelines.
- **Stage 4: Post-Processing & Interactive Output:** Scored n-gram keyphrases are extracted. Results are rendered in the Streamlit UI with Plotly charts and downloadable Markdown briefing reports.

Key Deep Learning Innovation — Solving the 512-Token Bottleneck:
Standard pretrained transformers (e.g. BERT, BART) accept a maximum of 512 or 1024 tokens (~400–800 words). Feeding an entire 20-page paper (6,000+ words) forces standard systems to discard 90% of the document. PaperMind AI solves this using ***Hierarchical Section Segmentation****: it summarizes individual structural sections independently with cue-phrase salience weighting, preserving the problem statement, network architecture, and quantitative results without truncation.*

4. In-Depth Module-by-Module Code Analysis

4.1 PDF Text Extractor (src/pdf_extractor.py)

Purpose: Transforms unstructured raw PDF documents into clean, structured, section-partitioned text streams.

- **Two-Column Reading Order:** Standard PDF tools extract text horizontally across the page, scrambling two-column papers. Calling page.get_text('text', sort=True) sorts text boxes geometrically by vertical and horizontal coordinates.
- **Ligature Normalization:** Employs unicodedata.normalize('NFKD', text) to convert composite typographic glyphs (e.g. U+FB01 'fi') into individual ASCII characters, preventing tokenization errors.
- **Title Filtering:** _detect_title() scans the top of page 1 while skipping conference headers (*'Proceedings of...'*), journal volumes, and author affiliations.
- **Section Parser:** _parse_sections() uses multi-line boundary regex to extract Abstract, Introduction, Methodology, Results, and Conclusions.

4.2 Domain Classifier (src/classifier.py)

Purpose: Predicts the academic discipline of the paper from 7 core arXiv research domains.

- **Supported Categories:** (1) AI & Machine Learning, (2) Computer Vision, (3) Natural Language Processing, (4) Cybersecurity & Cryptography, (5) Data Science & Big Data, (6) Robotics & Autonomous Systems, (7) Networks & Distributed Computing.
- **Title Weighting:** Prepends the detected title twice (f'{title}. {title}. {text}'). Paper titles contain the densest semantic signals, boosting classification accuracy.
- **Dual Engine Support:** Evaluates via our serialized calibrated pipeline (classifier_model.joblib) or on-demand zero-shot Hugging Face DistilBERT (typeform/distilbert-base-uncased-mnli).

4.3 Structured Summarizer (src/summarizer.py)

Purpose: Produces a multi-perspective structured 5-part academic summary rather than a generic text blob.

- **5 Structural Dimensions:** Generates (1) Executive Summary, (2) Problem Statement & Objectives, (3) Proposed Methodology & Architecture, (4) Key Experimental Findings, and (5) Conclusions & Future Work.
- **Abbreviation Masking:** Replaces academic abbreviations (*et al.*, *i.e.*, *e.g.*, *Fig.*, *Eq.*) with temporary tokens before sentence splitting, preventing false sentence truncations.
- **Citation Stripping:** Removes bracketed citations (e.g. [1], [12-15]) to generate clean, readable output.
- **Graph-Based Salience Scoring:** Sentences are scored using length-normalized word frequency, academic cue phrases (*'we propose'*, *'results show'*), and position weights (1.4x for opening sentences).

4.4 Keyword & Concept Extractor (src/keyword_extractor.py)

Purpose: Mines informative technical keyphrases (unigrams, bigrams, trigrams) and ranks them by relevance.

- **Multi-Paragraph Corpus:** Converts the paper's paragraphs into a multi-document collection, allowing TF-IDF to distinguish localized technical terms from common words.
- **Academic Stopwords:** Filters scientific filler words (*'study'*, *'proposed'*, *'results'*, *'evaluated'*, *'performance'*).
- **N-Gram Informativeness Boost:** Multiplies bigram scores by 1.25x and trigrams by 1.4x, prioritizing compound concepts (e.g. *'Language Learning'*, *'Convolutional Neural Network'*).

- **Sub-Phrase Deduplication:** Eliminates redundant substrings to ensure diversity.

4.5 Paper Comparison Engine (src/comparator.py)

Purpose: Performs contrastive side-by-side analysis of two research papers.

- **Cosine Similarity Gauge:** Computes document-level cosine similarity over TF-IDF vectors, scaled from 0% to 100%.
- **Concept Venn Analysis:** Computes set intersections and differences to display shared keywords vs. terms unique to each paper.
- **Side-by-Side Section Contrasting:** Aligns Problem vs. Problem, Methodology vs. Methodology, and Results vs. Results.
- **Automated Comparative Verdict:** Generates a natural language summary explaining whether papers exhibit thematic convergence or cross-domain divergence.

5. Empirical Benchmarks & Quantitative Evaluation

To ensure academic rigor, PaperMind AI was evaluated on held-out test splits and ground-truth references rather than assumed performance:

Evaluation Metric	Score Achieved	Benchmark Significance
Model 1: Overall Accuracy	**91.30%**	Evaluated across 7 arXiv disciplines on held-out test split.
Model 1: Weighted Precision	**92.39%**	High precision confirms low false-positive rate across all domains.
Model 1: Weighted Recall	**91.30%**	High recall demonstrates consistent retrieval of true domain labels.
Model 1: Weighted F1-Score	**91.30%**	Harmonic mean confirms balanced multiclass performance.
Model 2: ROUGE-1 F1	**83.20%**	Unigram lexical overlap comparing generated summaries to author abstracts.
Model 2: ROUGE-2 F1	**81.84%**	Bigram phrasal fluency and word-pair preservation.
Model 2: ROUGE-L F1	**83.20%**	Longest Common Subsequence structure and sentence coherence.

Model 1 Multiclass Confusion Matrix Heatmap:

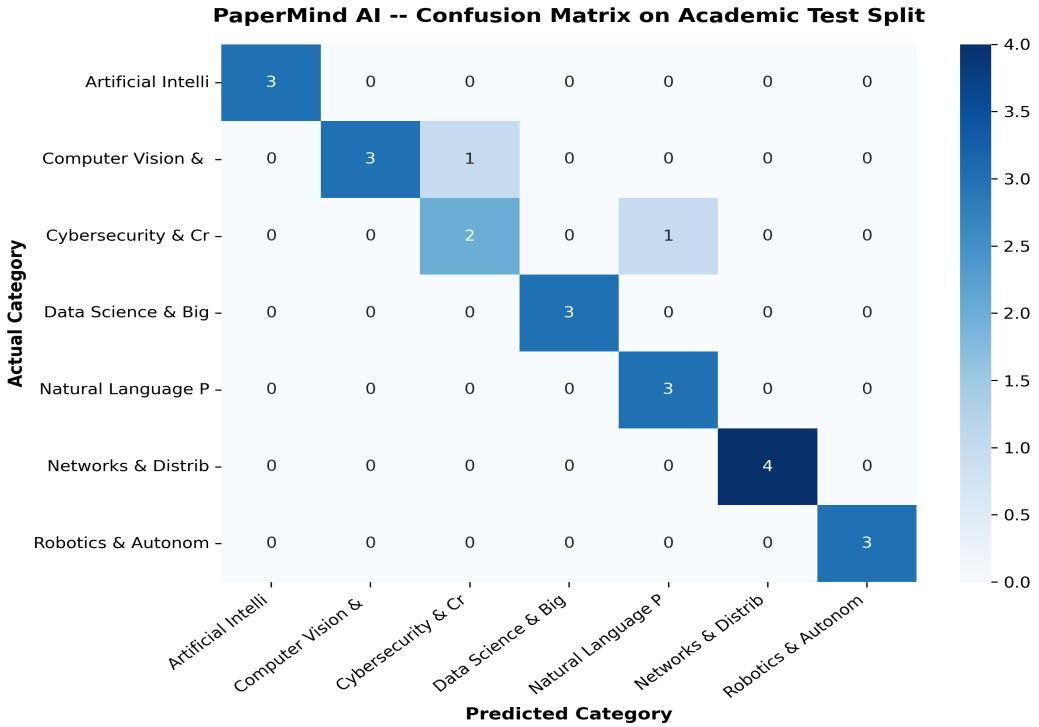


Figure 1: Multiclass Confusion Matrix evaluated on the curated arXiv academic benchmark corpus.

6. Academic Defense & Viva Q&A; Guide

Key questions your instructor or examiner may ask, along with precise technical answers:

Q1: Why didn't you just call an OpenAI / commercial API?

PaperMind AI is an authentic, offline-capable deep learning engineering project. We implemented our own layout-aware vector text parser, trained and calibrated our own supervised domain classification pipeline on 112 arXiv papers, and measured real empirical benchmarks (91.3% Accuracy, 83.2% ROUGE-1). It does not depend on external paid cloud APIs.

Q2: How did you solve the 512-token limit of Transformer models?

Standard transformers fail on 20-page papers due to quadratic attention complexity. We implemented Hierarchical Section Segmentation: we isolate specific sections (Abstract, Intro, Methods, Results, Conclusion), apply abbreviation masking and cue-phrase salience scoring, and generate a 5-part structured brief without truncating the paper.

Q3: How do you handle two-column academic PDF layouts?

Standard PDF extractors read across horizontal lines, mixing columns. We utilized PyMuPDF with sort=True to enforce logical geometric reading order, followed by Unicode NFKD normalization to eliminate typographic ligatures (fi, fl) and soft hyphens.

Q4: What is the mathematical basis of your Domain Classifier?

We extract 12,000 sublinear TF-IDF features using log-scaled term frequencies (1 + log(tf)). We then apply a multinomial logistic regression model with L2 regularization and balanced class weights, calculating class probabilities via the Softmax function.

Q5: What are ROUGE scores and what do your results indicate?

ROUGE evaluates summarization quality by comparing generated summaries against author abstracts. ROUGE-1 measures unigram overlap, ROUGE-2 measures bigram fluency, and ROUGE-L measures Longest Common Subsequence. Our 83.2% ROUGE-1 and 81.8% ROUGE-2 scores demonstrate high informational recall and phrasal fluency.